

Complete Exercise Manual

*Functional Programming Foundations
& Compiler Construction Laboratories*

October 04, 2026

Preface

This document serves as the comprehensive, self-contained **Exercise & Laboratory Problem Manual** to accompany the course monograph *Principles of Functional Programming & Compiler Construction*. It collects every theoretical exercise, derivation prompt, laboratory exercise, and incremental project milestone across:

1. All core functional programming foundations (Lambda calculus, ADTs, laziness, monads, and category theory).
2. Modern front-end compiler engineering (Alex lexers, Happy LALR(1) parsers, AST design, and evaluation environments).
3. Back-end compiler engineering and machine architectures (the 17-instruction virtual stack machine, code generation, and activation records).
4. The full 24-milestone Scientific Calculator and 42-milestone Pascal-to-Bytecode compiler specifications.

For the corresponding step-by-step solutions, formal proofs, and complete code listings, please consult the companion **Solutions Manual** (`solutions.pdf`).

Contents

Preface	i
I Part I: Functional Programming Foundations	1
1 Chapter 1: Paradigms of Computation & Lambda Calculus	1
2 Chapter 2: Algebraic Data Types & Recursive Data Modeling	1
3 Chapter 3: Evaluation Strategies & The Mechanics of Laziness	2
4 Chapter 4: Category Theoretic Abstractions: Functors, Applicatives & Monads . . .	3
II Part II: Solutions for Front-End Architecture & Syntax Analysis	4
1 Chapter 6: Lexical & Syntactic Analysis: Alex & Happy in Practice	4
2 Chapter 7: Abstract Syntax Trees & Expression Evaluation: The Calculator System .	4
III Part III: Solutions for Back-End Construction & Code Generation	5
1 Chapter 8: Stack Machine Architecture: The 17-Instruction Virtual Machine	5
2 Chapter 9: Code Generation: Compiling High-Level Languages to Stack Bytecode .	5

3 Chapter 10: Functions, Activation Records & Runtime Memory Management	6
4 Chapter 11: Advanced Topics: Type Checking, Optimizations & Native Code Generation	7
 IV Part IV: Course Laboratories & Semester Projects	 7
1 Problem Set 1: Lexing and Parsing Well-Parentthesized Expressions	7
2 Problem Set 2 & 3: Architectures for the 24 Calculator and 42 Pascal Milestones .	7

Part I: Functional Programming Foundations

Chapter 1: Paradigms of Computation & Lambda Calculus

Exercise 1 (Function Invocation Syntax). *[Ref: intro_haskell.pdf, Slide 5]*

Problem Statement:

How do you invoke the recursive factorial function `fact` with the integer argument `5` in Haskell? Explain the syntactic difference between Haskell's application convention and traditional imperative languages.

Exercise 2 (First-Principles Currying & Uncurrying). *[Ref: intro_haskell.pdf, Slide 9]*

Problem Statement:

Implement the foundational functions `curry` and `uncurry` from first principles without using standard library shortcuts:

```
curry    :: ((a, b) -> c) -> a -> b -> c
uncurry  :: (a -> b -> c) -> ((a, b) -> c)
```

HASKELL

Demonstrate that they form an isomorphism by showing that $\text{curry} \circ \text{uncurry} \equiv \text{id}$ and $\text{uncurry} \circ \text{curry} \equiv \text{id}$.

Chapter 2: Algebraic Data Types & Recursive Data Modeling

Exercise 3 (Polymorphic Constructor Inference). *[Ref: intro_haskell.pdf, Slide 8]*

Problem Statement:

Consider the polymorphic inductive data type:

```
data MyList a = Empty | Cons a (MyList a) deriving (Show)
```

HASKELL

What is the deduced type of the nested expression `Cons Empty Empty` in GHCi? Explain how the Hindley-Milner type inference engine resolves this term.

Exercise 4 (Typeclass Analysis: ``lookup``). *[Ref: intro_haskell.pdf, Slide 10]*

Problem Statement:

Analyze the full type signature of the association list lookup function:

```
lookup :: Eq a => a -> [(a, b)] -> Maybe b
```

HASKELL

Explain:

1. Why the typeclass constraint `Eq a` is mathematically required.
2. Why the return type is `Maybe b` rather than simply `b` or throwing an exception.

Exercise 5 (Laboratory TP 0: Binary Trees and Traversals). *[Ref: intro_haskell.pdf, Slide 18 (TP 0)]*

Problem Statement:

1. Define a polymorphic binary tree data type `Tree a` with empty leaves and value-bearing internal nodes.
2. Implement recursive functions computing:
 - The height of the tree (`height :: Tree a -> Int`).
 - The total number of nodes (`nbNodes :: Tree a -> Int`).
 - The number of leaves (`nbLeaves :: Tree a -> Int`).
3. Implement the three canonical depth-first tree traversals returning `[a]`: in-order, pre-order, and post-order.

Chapter 3: Evaluation Strategies & The Mechanics of Laziness

Exercise 6 (Infinite Sequence Generation via Laziness). *[Ref: intro_haskell.pdf, Slide 11]*

Problem Statement:

Implement the infinite sequence of natural numbers `[1, 2, 3, 4, ...]`:

1. Using an explicit recursive generator function.
2. Using Haskell's core lazy list primitives (`iterate` and arithmetic sequences).

Exercise 7 (Higher-Order Consecutive Element Sums). *[Ref: intro_haskell.pdf, Slide 12]*

Problem Statement:

Write a point-free or lambda expression taking a list $l = [x_0, x_1, x_2, \dots]$ and returning the pairwise consecutive sums $[x_0 + x_1, x_1 + x_2, \dots]$. Analyze the typing and behavior of its constituent higher-order functions.

Exercise 8 (Quicksort and Operator Sections). *[Ref: intro_haskell.pdf, Slide 14]*

Problem Statement:

In the canonical Haskell Quicksort implementation:

```
qs :: Ord a => [a] -> [a]
qs []      = []
qs (p:tl) = (qs $ filter (< p) tl) ++ [p] ++ (qs $ filter (>= p) tl)
```

HASKELL

Explain the syntax `(< p)` and `(>= p)`. What are operator sections, and why is parentheses placement crucial?

Chapter 4: Category Theoretic Abstractions: Functors, Applicatives & Monads

Exercise 9 (Formal Proof of Monad Laws for `Maybe`). [Ref: *intro_haskell.pdf*, Slide 21]

Problem Statement:

For the standard `Maybe` monad instance:

```
return x      = Just x
Nothing >=> _ = Nothing
Just x >=> f = f x
```

HASKELL

Prove analytically that the 3 categorical monad laws are satisfied for all values:

1. **Left Identity:** `return a >=> f` $\equiv$ `f a`.
2. **Right Identity:** `m >=> return` $\equiv$ `m`.
3. **Associativity:** `(m >=> f) >=> g` $\equiv$ `m >=> (\x -> f x >=> g)`.

Exercise 10 (Peeling Monadic Structure (`join`)). [Ref: *intro_haskell.pdf*, Slide 24]

Problem Statement:

The flattening operation (peeling one monadic layer) is defined as:

```
join :: Monad m => m (m a) -> m a
```

HASKELL

1. Implement `join` specifically by hand for `Maybe` (`Maybe a -> Maybe a`).
2. Implement `join` universally for any arbitrary `Monad m` using only `(>=>)` and `id`.

Exercise 11 (List Monad Desugaring & Sequential I/O Printing). [Ref: *intro_haskell.pdf*, Slide 26]

Problem Statement:

1. Desugar the following list comprehension `do`-block into explicit `(>=>)` and `return`, verifying the typing at each step:

```
do { x <- [1, 2]; y <- [1, 5]; return (x, y) }
```

HASKELL

2. Implement a function `printAll :: [String] -> IO ()` that prints each string on a separate line using only pure recursion or monadic bind, without using `mapM_` from Prelude.

Part II: Solutions for Front-End Architecture & Syntax Analysis

Chapter 6: Lexical & Syntactic Analysis: Alex & Happy in Practice

Exercise 12 (Parenthesized Expression Parsing & Max Depth). [Ref: *compil_happy.pdf*, Section 4.5 (paren-01, paren-02)]

Problem Statement:

1. Write a Happy grammar over tokens `'('` and `')'` that parses well-parenthesized expressions and returns the maximum nesting depth directly as an integer.
2. Explain how source position tracking via Alex's `%wrapper "posn"` enables accurate error diagnostics.

Exercise 13 (Lexical Identifiers in Context-Free Grammars). [Ref: *compil_happy.pdf*, Section 4.5 (paren-04, paren-05)]

Problem Statement:

In `paren-04`, the Happy grammar allows identifier tokens only at leaf positions:

```
Expr : {- empty -}      { Empty }
      | ident           { Leaf $1 }
      | '(' Expr ')' Expr { Node $2 $4 }
```

HASKELL

1. Where are identifiers forbidden by this grammar? Give an example of an invalid string.
2. How can the grammar be refactored to allow arbitrary sequences of identifiers and parenthesized groups?

Chapter 7: Abstract Syntax Trees & Expression Evaluation: The Calculator System

Exercise 14 (Pure Arithmetic AST & Evaluator). [Ref: *compil_happy.pdf*, Section 3.1–3.2]

Problem Statement:

1. Define a pure algebraic data type `AExpr` supporting floating-point literals (`Double`), unary negation, and the binary operators `+`, `-`, `*`, `/`, and `^`.

2. Implement an evaluator `eval :: AExpr -> Maybe Double` that safely returns `Nothing` when encountering a division by zero.

Exercise 15 (Environment-Based Evaluation & Variables). [Ref: *compil_happy.pdf*, Section 3.4 & Section 5]

Problem Statement:

Extend `AExpr` with variable names `Var String` and assignments `Assign String AExpr`. Formulate an evaluator `evalEnv :: [(String, Double)] -> AExpr -> Either String (Double, [(String, Double)])` that looks up bound variables, rejects unbound identifiers with a descriptive error message, and updates the environment upon assignment.

Part III: Solutions for Back-End Construction & Code Generation

Chapter 8: Stack Machine Architecture: The 17-Instruction Virtual Machine

Exercise 16 (Relational Comparison on the 17-Instruction Machine). [Ref: *defmachine.pdf*, Slide 9]

Problem Statement:

The virtual machine does not possess a primitive comparison instruction (such as `<` or `CMP`). It provides only `BEZ` (branch if zero) and `BGZ` (branch if strictly positive). Given two values `A` and `B` at the top of the stack (with `B` above `A`), write a minimal symbolic assembly sequence computing the boolean test `A < B`, leaving 1 on the stack if true and 0 if false.

Exercise 17 (Manual Assembly Compilation: Euclidean GCD). [Ref: *happy_pascal.pdf*, Milestone 14 & *defmachine.pdf*]

Problem Statement:

Write the complete symbolic assembly program calculating the Greatest Common Divisor (`gcd`) of two integers read from standard input, using the Euclidean algorithm, and printing the result to `OUT`.

Chapter 9: Code Generation: Compiling High-Level Languages to Stack Bytecode

Exercise 18 (Recursive AST to Stack Bytecode Emission). [Ref: *happy_pascal.pdf*, Milestones 1–6]

Problem Statement:

Consider an arithmetic expression AST type:

```
data Op = Add | Sub | Mul | Div
data Expr = Lit Int | Var String | BinOp Op Expr Expr
```

HASKELL

Write a pure code generation function `compileExpr :: Expr -> [String]` that emits assembly instructions maintaining the stack invariant (leaving exactly one result at the top of the operand stack).

Exercise 19 (Array Indexing L-Value & R-Value Emission). [Ref: *happy_pascal.pdf*, Milestone 15 & *defmachine.pdf*]

Problem Statement:

1. Specify the stack assembly sequence to read the value of `primes[i + 1]`.
2. Specify the stack assembly sequence to execute the assignment `primes[i + 1] := 42;`.

Chapter 10: Functions, Activation Records & Runtime Memory Management

Exercise 20 (Recursive Function Activation & Factorial Emission). [Ref: *happy_pascal.pdf*, Milestone 28 & *intro_haskell.pdf*, Slide 5]

Problem Statement:

Outline the stack machine calling convention for the recursive function:

```
function fact(n: integer): integer;
begin
  if n <= 1 then fact := 1 else fact := n * fact(n - 1)
end;
```

PASCAL

Provide the symbolic assembly implementation with return address preservation and frame teardown.

Exercise 21 (Fibonacci: Naive Tree Recursion vs. Linear Accumulator). [Ref: *happy_pascal.pdf*, Milestone 28]

Problem Statement:

1. Write the naive recursive Fibonacci function $F(n) = F(n - 1) + F(n - 2)$ and analyze its stack frame growth.
2. Implement an optimized linear tail-recursive accumulator version in Haskell and explain why it compiles to $\mathcal{O}(1)$ stack memory.

Chapter 11: Advanced Topics: Type Checking, Optimizations & Native Code Generation

Exercise 22 (Constant Folding AST Optimization). [Ref: *happy_pascal.pdf*, Milestone 41]

Problem Statement:

Implement a recursive optimization pass `foldConstants :: Expr -> Expr` on arithmetic expressions that collapses subtrees containing only constant integer literals into single literal nodes at compile time.

Exercise 23 (Short-Circuit Boolean Evaluation). [Ref: *happy_pascal.pdf*, Milestone 38]

Problem Statement:

Pascal's logical operators `and` and `or` can cause unwanted side effects or division-by-zero crashes if both operands are eagerly evaluated (e.g. `(y <> 0) and (x mod y == 0)`). Formulate the conditional jump pattern that guarantees short-circuit execution on the stack machine.

Part IV: Course Laboratories & Semester Projects

Problem Set 1: Lexing and Parsing Well-Parentthesized Expressions

Exercise 24 (Parenthesis Depth Parser). [Ref: *compil_happy.pdf* (paren-01)]

Problem Statement:

Write a Happy grammar that directly computes the maximum nesting depth of well-parentthesized expressions as an integer.

Exercise 25 (Rose Tree Parse Generation). [Ref: *compil_happy.pdf* (paren-03)]

Problem Statement:

Modify the grammar to produce an explicit multi-way tree (generalized rose tree) representing the nested syntactic hierarchy.

Problem Set 2 & 3: Architectures for the 24 Calculator and 42 Pascal Milestones

Exercise 26 (Incremental Calculator Pipeline Architecture (24 Milestones)). [Ref: *compil_happy.pdf*, Section 5]

Problem Statement:

How should the scanner (`CalculatriceLexer.x`), parser (`CalculatriceParser.y`), and runtime REPL (`Calcul.hs`) be structured to systematically satisfy all 24 milestones (from Double numbers to Unicode variables and implicit multiplication)?

Exercise 27 (Pascal Compiler Code Generator Architecture (42 Milestones)). [Ref: *happy_pascal.pdf*, Section 2]

Problem Statement:

Provide the overall architecture for the 42-stage Pascal compiler translating Pascal ASTs into 17-instruction symbolic bytecode, specifically addressing label generation, memory reservation, and activation records.